

1. Briefly describe the artifact. What is it? When was it created?

The artifact selected for the Software Engineering and Design category was originally developed in IT145: Found in App Development. This is a Java-based rescue animal management application designed to support the intake, management and reservation of rescue animals. The application uses object-oriented programming concepts including classes, inheritance, encapsulation and collections. The application contains a base `RescueAnimal` class with `Dog` and `Monkey` subclasses and uses Java `ArrayList` collections to store animal information. The original application provides a functional foundation, but much of its functionality is concentrated in the `Driver` class. The `Driver` class displays the user interface, collects and validates input, creates animal objects, searches through the collections, processes reservations and directly manages the application's data. This was created early on in my academic journey during the spring semester of 2025, so it was interesting to see how far I've progressed as a programmer since then.

2. Justify the inclusion of the artifact in your ePortfolio. Why did you select this item?

What specific components of the artifact showcase your skills and abilities in software development? How was the artifact improved?

I selected this artifact for my ePortfolio because it provided a strong opportunity to demonstrate software design skills and the ability to think about an application as a complete system rather than as a collection of individual features. The original artifact had most of its functionality consolidated in a single `Driver` class, which made the application more difficult to understand, maintain and expand upon.

This provided an opportunity to refactor the application using layered architecture and separation of concerns. Responsibilities that were previously combined in the Driver class were separated into the user interface, business logic and data-access layers. The addition of the Animal Repository interface also allows the application to support different storage implementations without requiring major changes to the business logic or user interface.

The artifact was improved through several smaller code and functionality changes, but the most significant improvement was the architectural redesign. The new structure demonstrates object-oriented design principles, dependency inversion, reduced coupling and a more maintainable and readable codebase. It also provides a foundation for future expansion, including the addition of a database without requiring the application to be completely overhauled.

3. Did you meet the course outcomes you planned to meet with this enhancement in Module One? Do you have any updates to your outcome-coverage plans?

I met the course outcomes that I originally planned to address in Module One, and I did not make any changes to my original outcome-coverage plan. The enhancement continues to align with following outcomes:

“Employ strategies for building collaborative environments that enable diverse audiences to support organizational decision-making in the field of computer science.”

The redesigned architecture establishes clear responsibilities between application components and reduces unnecessary coupling. This supports collaborative development by allowing developers to work within defined areas of responsibility and makes changes with less impact on unrelated components. The architecture also makes design decisions easier to understand and evaluate. The accompanying code review and written narratives provide a common representation of the applications structure and reasoning behind the architectural changes, supporting informed decisions about future maintenance and expansion.

“Design, develop, and deliver professional-quality oral, written, and visual communications that are coherent, technically sound, and appropriately adapted to specific audiences and contexts.”

The enhancement is supported by technical documentation and an oral code review. These materials explain the limitations of the original application, the new layered architecture, the responsibilities of each layer and the reasoning behind the design decisions. The information can be communicated at different levels of technical detail, allowing implementation details to be emphasized for developers while the practical benefits of improved maintainability and extensibility can be communicated to broader stakeholders.

4. Reflect on the process of enhancing and modifying the artifact. What did you learn as you were creating it and improving it? What challenges did you face?

One of the biggest challenges during this enhancement was balancing the desire to improve the application with the need to keep the scope focused on the primary architectural goal. There were many areas where the original application could have been changed or expanded, but I had to determine which changes contributed to the enhancement and which would add unnecessary complexity. My goal was to preserve the original functionality while improving the structure of the application rather than completely replacing the original implementation with a different design.

This process also reinforced the value of making changes incrementally rather than trying to build the finished architecture all at once. Separating the application into layers required moving responsibilities gradually and making sure the existing functionality continued to work as the structure changed. This approach made it easier to understand the effect of each change and helped prevent the refactoring from becoming an unmanageable rewrite.

Another important learning experience was designing applications with future development in mind. I had not previously worked with an interface serving as an abstraction between business logic and data storage. Learning how the Animal Repository interface could support the current in-memory implementation while also providing a path for a future JDBC and MySQL implementation helped me better understand dependency inversion and the value of designing software around abstractions rather than specific implementations.

There were also several design trade-offs that I had to consider. For example, the intake process does not determine whether an animal name is a duplicate until after all the animal's information has been entered. Moving the duplicate-checking logic into the service layer keeps business rules out of the user interface, but it also means the application does not provide immediate feedback before the user finishes entering the animal's information. I chose to accept this limitation because maintaining a clear separation between the user interface and business logic was more important to the goals of this enhancement than adding additional complexity to the console interaction.